

Basic part 2 (venv setup) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (the proper terminal workflow, with each step verified). Read Basic first even if you already know venvs — it's the fastest way to spot exactly what Intermediate adds.

Where this exercise is saved: `practice/venv_setup_practice.md` — notes, not a script. Every block below is a terminal command, so there is nothing to `python -run`: keep a Markdown file with each command you ran and the output you got back, and you will have your own record of the sequence the next time you start a project.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This exercise is about typing commands, not writing code. You need to do four things in order: make a new, empty Python space just for this project, turn it on, install one package into it, and write down what you installed.

Here are the four commands, in order:

1. `python -m venv .venv` — makes the new space.
2. `source .venv/bin/activate` — turns it on (Windows: `.venv\Scripts\activate`).
3. `pip install requests` — installs one package.
4. `pip freeze > requirements.txt` — writes down what's installed.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The mental model: `python -m venv .venv` creates an isolated Python environment (its own `site-packages` folder, separate from your system Python). Activating it (`source .venv/bin/activate`) changes which `python` and `pip` your shell resolves to, for the rest of that terminal session. From that point, anything you `pip install` only affects this project, not your whole system or any other project.

`pip freeze > requirements.txt` writes every installed package and its exact version to a file — this is what lets anyone (including future you) recreate the identical environment later with `pip install -r requirements.txt`.

Same four commands, with what to check after each one:

1. `python -m venv .venv` — check: a new `.venv/` folder now exists.
2. `source .venv/bin/activate` — check: your terminal prompt now shows `(.venv)` at the start.
3. `pip install requests` — check: `pip list` shows `requests` in the installed packages.
4. `pip freeze > requirements.txt` — check: opening `requirements.txt` shows a line like `requests==2.32.3`.

If step 2 doesn't seem to do anything visible, that's the most common place this goes wrong — see the [Solution](#) for what to check.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
open a terminal, go to your project folder
create a venv named .venv
turn it on
install "requests" with pip
save the list of installed packages to requirements.txt
open requirements.txt and look for "requests" in it
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# → practice/venv_setup_practice.md
python -m venv .venv
source .venv/bin/activate
pip install requests
pip freeze > requirements.txt
```

Expected output: none of these 4 commands print much on their own — `python -m venv .venv` and `source .venv/bin/activate` are silent when they succeed, and `pip install requests` prints a few lines ending in something like `Successfully installed requests-2.32.3 ...`. Run `cat requirements.txt` afterward to actually see a result — see the Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
in your project folder:
python -m venv .venv
source .venv/bin/activate      (or .venv\Scripts\activate on Windows)
pip install requests
pip freeze > requirements.txt

confirm:
prompt shows (.venv)
requirements.txt contains a line starting with "requests=="
```

The commands, plus how to check each result:

```
# → practice/venv_setup_practice.md
python -m venv .venv
ls .venv                        # should show bin/, lib/, etc.

source .venv/bin/activate
which python                    # should now point inside .venv/

pip install requests
pip show requests               # confirms it installed correctly

pip freeze > requirements.txt
cat requirements.txt            # should list requests==<version>
```

Expected output of the last line, `cat requirements.txt` :

```
certifi==2024.8.30
charset-normalizer==3.3.2
idna==3.8
requests==2.32.3
urllib3==2.2.3
```

(Exact version numbers will differ depending on when you run this — `pip freeze` also lists `requests` 's own dependencies, not just `requests` itself, which is why you see 5 lines instead of 1.)

If activation seems to silently do nothing, see the [Solution](#) for the most common cause.

Difference between Basic and Intermediate: same 4 underlying actions at 2 completeness levels, shown here both as pseudocode and as near-complete code — Basic proves you can do them by hand, Intermediate adds a way to confirm each step actually worked.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)